Servo Motor Door Lock

CS 435 Embedded Systems

Montserrat Jara, Beau Dougan, Theo Garvey

Overview

Abstract:

For our project we connected multiple components to create a door lock application. For this project we used a nucleo board, a servo motor, a 4x4 keypad, a buzzer, a red led, a green led, and a small wooden door. For this project we used these components to reflect real world applications for embedded systems. Our system has a single focus to unlock and lock a door using a keypad. Many people have opted to use digital door locks and embedded door lock systems as opposed to a traditional key and lock. Our project allowed us to use the skills we learned about embedded systems to create our own door lock and we used a servo to unlock and lock the door. Our system uses the digital pins and pwm pins of our nucleo board to connect all our components. Additionally we used a bread board to avoid soldering our components.We were inspired by a similar project online and we simplified the components for our project. If we had more time to work on this project we would like to incorporate more components such as the bluetooth module to reflect the internet of things (IOT) and embedded systems.

Introduction:

The objectives of this project are to use a nucleo board, a buzzer, a servo motor, and a 4x4 keypad to simulate a keypad doorlock. When correct codes are entered, the servo should rotate to ‘open’ the lock if the current state is locked, and when incorrect codes are entered, the servo should rotate to ‘close’ the lock if the current state is unlocked. If the device is set to change into the same state, no change will happen. Additionally, when correct codes are entered, the green LED should turn on and the buzzer will play a noise to signify a correct code being entered, and when incorrect codes are entered, the red LED should turn on and the buzzer will play a noise to signify an incorrect code being entered. The scope of this project includes using pwm outputs for the buzzer and motor, digital outputs for our LEDs and digital inputs for our key pad. The relevance of this project to real world applications would be door locks. Our project demonstrates how we can use the above mentioned components to show how a keypad can be used to lock and unlock a door.

Similar work:

A similar project online [1] used a bluetooth sensor to send a signal via a smart phone to the arduino board which rotates the servo and latches or unlatches a lock on a door. The project we referencing used two different buttons on their smart phone to send a value of 0 or 90 which represents an angle for the servo motor to rotate to. For our project we decided to not include a smart phone or bluetooth module and instead use a 4x4 keypad matrix. Similar to the project mentioned we also used a servo and servo library to lock and unlock our door. One small difference is we used the write function to send values between 0.0 - 1.0. We found through testing that the range of motion of our servo motor was between 0.0 - 0.95. We also decided to go with a 4x4 matrix because we thought it was relevant to real world applications and we thought a keypad would allow for the user to enter their password as opposed to clicking a button on their phone. Another difference is that instead of using an arduino uno board we will be using the nucleo board. Lastly, we plan on using a red and a green led as well as a buzzer to represent when correct or incorrect codes are entered.

Conceptual design of the system:

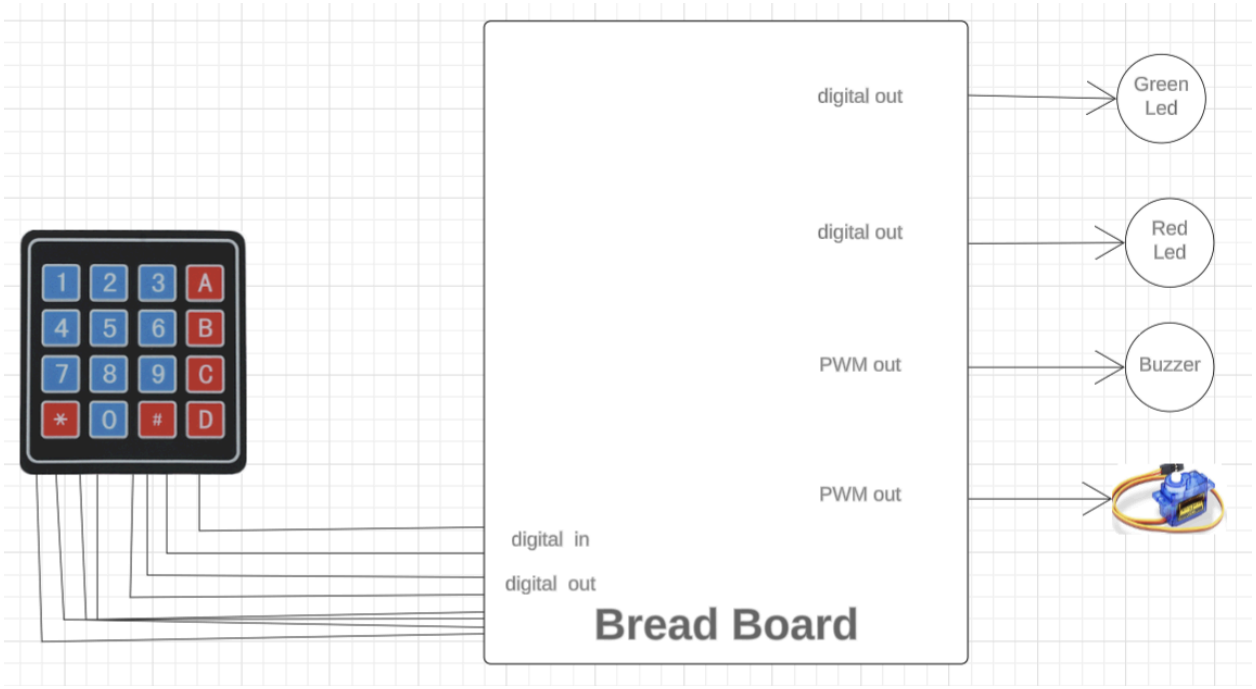

Figure 1: Block Diagram

In the block diagram above we use a bread board to connect all the components together. What is not shown is that the bread board is connected to the nucleo board. We connect ground and 3.3V to the bread board which powers our components. Our leds are connected to digital out pins on the nucleo board as well as ground and voltage. The buzzer is connected to ground and to a pwm pin. Additionally the servo motor is also connected to a pwm pin as well as ground and voltage. The keypad is connected to eight digital pins on the nucleo board; the first four pins, which represent rows on the keypad, are passed through the breadboard with a pull up connection and are configured as digital inputs on the nucleo board and last four pins, which represent columns on the keypad, are configured as digital outputs.

Implementation details

Hardware design/setup:

One thing that determined the way we placed wires and components onto the bread board was the lack of wires. This made it so most of the wires that connect to the board had to be as close to the nucleo board as possible. Given how many resistors there were and that their wires could not be touching it required a lot of thought to how things would have to be placed onto the board. The Buzzer is connected to D5 on the nucleo board. The servo is connected directly into the nucleo board without passing through the breadboard like all the other components; it is connected to 5V, GND, and D6. The leds are connected to D4 and D2 on the nucleo board. The keypad is connected to 8 different ports: PC_0, PC_1, PC_2, PC_3, PC_4, PC_10, PC_11, and PC_12. The first four pins of the keypad(R1, R2, R3, and R4) served as the input ports of the keypad. To prevent floating inputs, we added 2.2 kΩ resistors to each of the input ports, which were used as a pull-up on the breadboard, and then they were connected to PC_4, PC_10, PC_11 and PC_12. The other four pins of the keypad(C1, C2, C3, and C4) served as the output ports to the microcontroller, which we directly connected to the Nucleo board in pins PC_0, PC_1, PC_2 and PC_3.

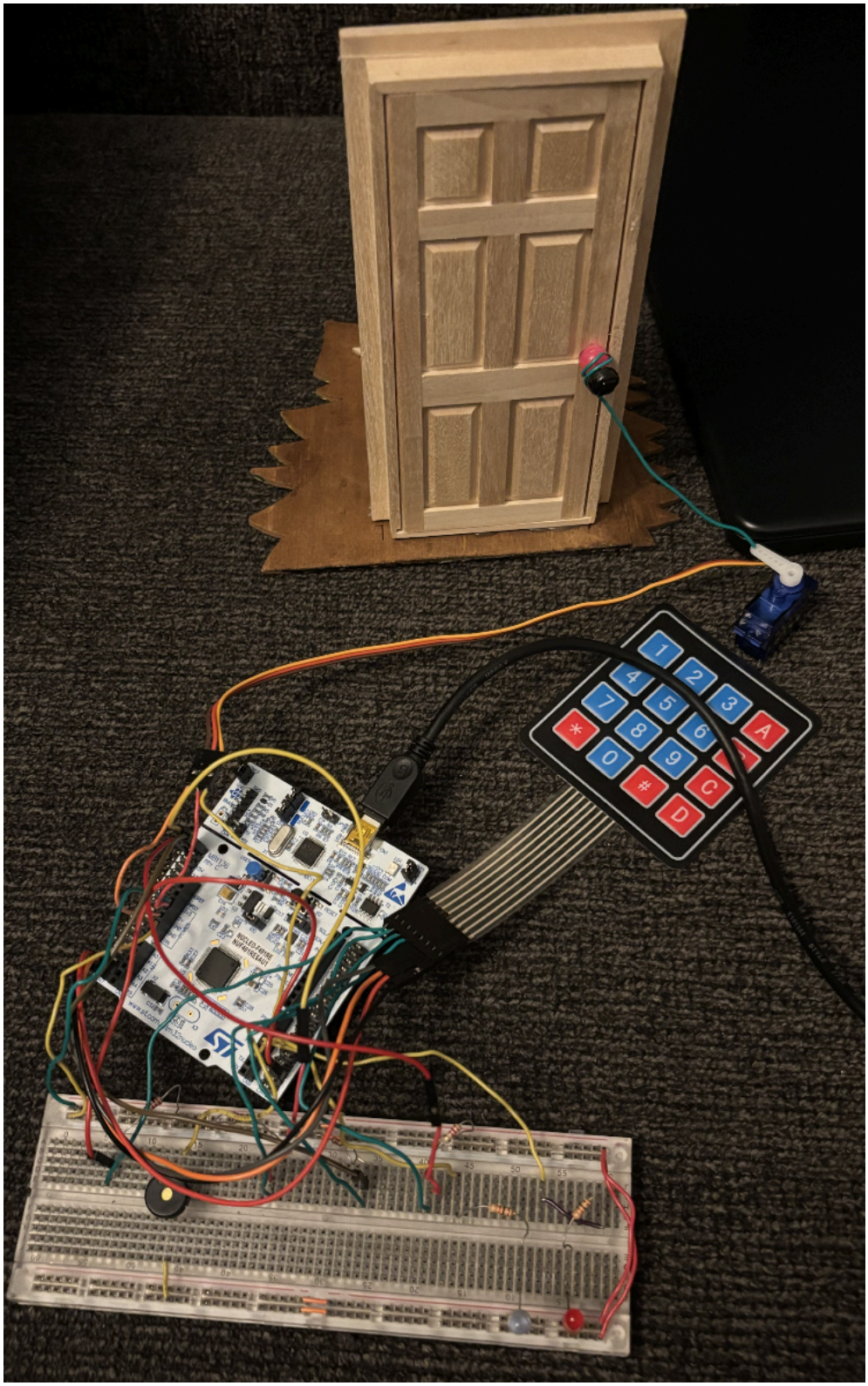

Figure 2: Hardware Set up

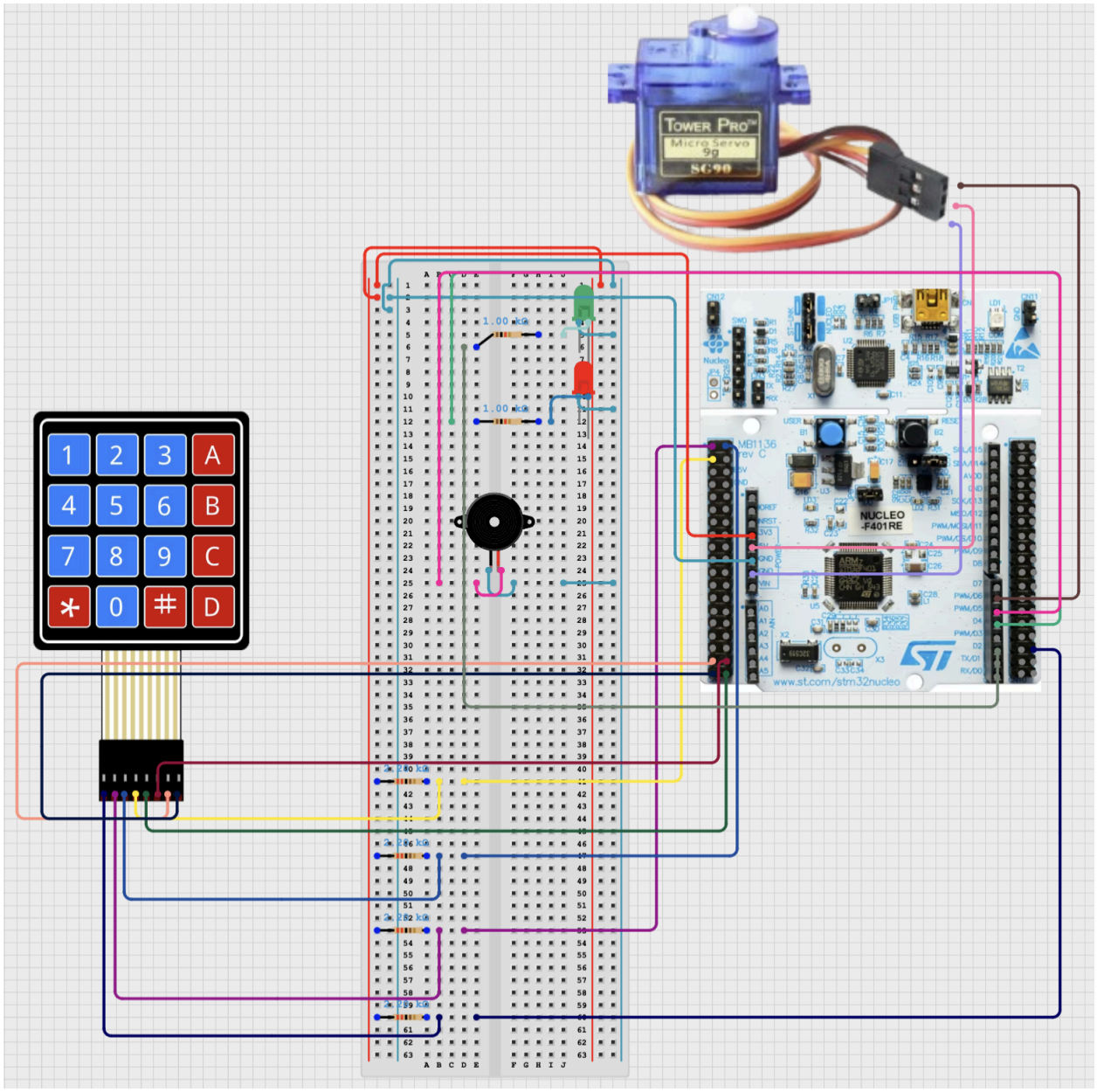

Figure 3: Circuit Diagram

Code implementation:

Before the loop each component is configured. The leds are defined as digital outputs. The servo is configured as a servo using the Servo library [7] which registers it as a PWM output. The Matrix keypad is configured using the keypad.h library [6] which registers the row pins as digital in and the column pins as digital out. These row pins are connected to 3v3 by way of 2.2k ohm resistors and are configured to pull up. The buzzer is configured as a PWM output. The lock will always initialize as engaged. Inside of the infinite while loop to begin each loop we set the string which stores the combination the user enters to empty. In another nested while loop which runs until 4 characters are entered into the combination string it checks if a key is pressed then and also is not the '#' key which is used as a lock button. When a valid key is pressed that key is appended to the combination string. If no key is pressed nothing will be done

and the loop will repeat until 4 characters are entered. If the '#' key is pressed or 4 characters are now in the combination string the loop is broken. Then there is an if statement which checks the code entered with the correct code. If they are the same it checks the locked flag. If locked then the servo will move to the unlocked position, turn on the green led and have a buzzer play a noise to symbolize a correct combination has been entered. If already unlocked then the buzzer will play but the other parts of the code will not. If the code entered was incorrect, or a '#' was pressed which is a digit that is not part of the correct combination so it will be incorrect every time, then the code checks if the servo is currently locked. If unlocked the servo will move to the locked position, the red led will turn on and the green led will turn off, finally the buzzer will play a noise which symbolizes an incorrect code being pressed. If currently locked the only step that will activate is the incorrect code noise on the buzzer being played. Then the device will reenter the infinite loop again. In the code there are functions to play sounds, and also for the locking and unlocking steps.

PDL:

BEGIN
Configure RED LED as digital output
Configure GREEN LED as digital output
Configure Servo as Servo using servo library[7] (PWM pin)
Configure Matrix 4x4 Keypad as Keypad using the keypad.h library [6]
Configure Buzzer as PwmOut
Initialize lock as engaged
DO FOREVER
Set string to store combination to an empty string
DO WHILE COMBINATION IS LESS THAN 4 CHARACTERS
IF a key is pressed **AND** key is not '#' **THEN**
Append that key to the combination string
ELSEIF key pressed is '#'
BREAK
ELSE
Do nothing
ENDDO
IF 4x4 Keypad code is correct **THEN**
Have servo move to unlocked position simulating the lock disengaging or not changing if already unlocked
Turn on green led and turn red led off
Have buzzer play a high pitched noise
ELSE
Have servo move to locked position simulating the lock engaging or not changing if already locked
Turn on red led and turn green led off
Have buzzer play a low pitched noise
ENDDO
END

Experimental results

Testing Correct Code while door is locked:

Expected output = door opens, buzzer beeps with correct code sound, red led turns off, green led turns on
Actual output = door opens, buzzer beeps with correct code sound, red led turns off, green led turns on

Testing correct code while door is unlocked:

Expected output = buzzer beeps with correct code sound
actual output = buzzer beeps with correct code sound

Testing Incorrect Code while door is unlocked:

Expected output = door closes, red led turns on, green led turns off, buzzer beeps with incorrect sound
Actual output = door closes, red led turns on, green led turns off, buzzer beeps with incorrect sound

Testing Incorrect Code while door is locked:

Expected output = buzzer beeps with incorrect sound
Actual output = buzzer beeps with incorrect sound

Testing lock button (#) on keypad while door is unlocked:

Expected output = door closes, red led turns on, green led turns off, buzzer beeps with incorrect sound
Actual output = door closes, red led turns on, green led turns off, buzzer beeps with incorrect sound

Testing lock button (#) on keypad while door is locked:

Expected output = buzzer beeps with incorrect sound
Actual output = buzzer beeps with incorrect sound

Testing Servo write 0.0:

Expected output = arm points all the way to the left
Actual output = arm point the left but slightly off

Testing servo write 1.0:

Expected output = arm points all the way to the right
Actual output = arm points to the right but slightly off (0.95)

Lessons learned

Project management:

For our project management it was difficult to meet outside of class due to each member's outside responsibilities such as other courses, capstones, and work. However to overcome this issue we communicated frequently over messages in a group chat. We also divided up the tasks so that different members could work on separate components at the same time. Theo worked on the connections needed for the 4x4 matrix keypad. We had some issues with our original keypad which was an adrafruit keypad and we ultimately had to borrow a keypad from the lab for our project. Theo also created the code for our red and green led as well as our buzzer. Montse worked on the connections and code for the servo motor as well as the small door to demo the final project. Beau worked on the code for the keypad and worked on the code to connect all of the components for the final project. All members worked on the documentation for the first draft, final draft, and presentation and Beau created the video demonstration.

External learning resources:

We used many external resources such as the datasheets for all of the components used in our project. The servo data sheet was used to determine the connections to the nucleo board [3]. The servo data sheet provided information on how the motor received frequencies to rotate the motor 180 degrees. We also used the mbed website for information on an mbed library for the servo which allowed us to use a write function to easily rotate the motor[5]. The lab file provided information on where exactly to wire the keypad ports to and which pinouts would be the input or output ports[9]. We utilised the morpho connectors on the Nucleo board instead of the Arduino connectors. The PC_10, PC_11, and PC_12 are morpho connectors that were required for our input ports. The only way to connect to those three was through the male pin head, CN7.

In addition to the online resources we used we also relied on the lab located in the Viasat building to provide us extra wires to connect all our components.

Challenges faced by each team member:

Montse:

Challenges I faced related to the servo motor. When first connecting the servo motor I wasn't aware there was a servo mbed library. I initialized the servo as a pwm out and experimented with the frequencies I sent to the motor. I was frustrated with the range of motion on my motor and I thought it was due to the frequencies I was sending. However when I discovered I could use the servo motor mbed library I still ran into the same issue. My conclusion is that the servo due to use has a limited range of motion. This was easier to conclude after testing the motor using the mbed library. Additionally I struggled to find a way to demonstrate our project and decided to use a small door instead of a metal latch like the project we referenced.

Theo:

The challenge that I dealt with was figuring out how I would connect the keypad to the Nucleo board. I've had previous experience using the Nucleo Board and the 4x4 matrix keypad in a previous

class, so I referred to some previous coursework to wire the keypad. I struggled at first because of the first keypad that we used. It was a different brand from what I used back in my other course, which was a Parallax 4x4 matrix keypad. The keypad that we attempted to use first was an Adafruit 4x4 matrix keypad. As previously mentioned, the keypad has 4 input ports and 4 output ports. The main difference between the Parallax keypad and the Adafruit keypad was that the pinouts were switched, with the R ports being the first four ports and the C ports being the latter[8]. I attempted to switch the wiring by having the last 4 ports be our input ports, and the first 4 ports be our output ports. We ended up switching to the Parallax keypad since the Adafruit keypad was not effective.

Beau:

The main challenge for me was figuring out how to fit all the components onto the breadboard. We were taught in class how dangerous it is to have resistors wires touch each other so I made sure to be very careful to place them far apart. Another challenge was the lack of wires I had in my kit. When combining all the different components I had run out of the long green wires, even before using one to connect the servo to the door, so it was necessary to place the components strategically so that they would be able to reach the nucleo board. Also due to the lack of wires it was impossible to color coordinate the wires for each component which made bug testing much more difficult than it needed to be. When connections to the nucleo board were wrong I had a lot of difficulty figuring out the problem due to there being so many wires of various different colors.

Future work:

If we had more time to work on the project it would have been nice to incorporate a bluetooth component. We learned about the internet of things and how more embedded systems increase their usefulness by being able to exchange data over the internet. It may be useful to connect our system to the internet to track the amount of time the lock was opened that day. Additionally, if the user wanted to unlock the door via their smart phone instead of using the keypad, connecting our system to bluetooth would allow them to do so. The user may want to ensure they are the only person entering the code to unlock the door so they may want to track it. Additionally they may want the option to lock and unlock their door remotely. Another thing we would like to add in the future is the ability for the user to change the code on the door. Currently the door can only be opened with the key that is set up in the code which is currently “1234”.

Summary:

To summarize, this project uses many different components combined with a nucleo board to simulate a door lock which takes a key code as an input. Through trial and error we ended up with a door to demo the product, which acts similar to how you would be buzzed into a door of an apartment building. We combine many components, some of which we had hands-on experience with in this course such as the buzzer and the leds. Another component we learned about in class but did not have a chance to use is the servo. Then we added in a component we did not touch on in class which was the keypad. Through this project we got to combine many different sections of this course and turn it into a project. We also got valuable experience working with new components as well as how to research a new component to figure out how it connects to the board being used and how to write code for it.

Code listings:

```
#include "mbed.h"
#include "mbed_thread.h"
#include <string>
#include <Keypad.h>
#include <Servo.h>
#include <Buzzer.h>
// set up leds
DigitalOut green(D2);
DigitalOut red(D4);

//Set up servo
//range of servo is from 0.0 - .95
Servo locker(D6);

//initialize buzzer
static PwmOut buzzer(D5);

//initialize keypad
Keypad kpad(PC_0,PC_1,PC_2,PC_3,PC_4,PC_10,PC_11,PC_12);

//intialize variable locked which can tell you if the lock is currently locked or
unlocked
bool locked;

void play_correct_sound() {
   buzzer.period(1.0 / NOTE_A4);  // 440 Hz
   buzzer.write(.5);
   ThisThread::sleep_for(500ms);
   buzzer.write(0);
}

void play_incorrect_sound() {
   buzzer.period(1.0 / NOTE_E2);   // low pitch
   buzzer.write(.5);
   ThisThread::sleep_for(200ms);

   buzzer.period(1.0 / NOTE_C2);   // even lower
   ThisThread::sleep_for(300ms);

   buzzer.write(0);
}
```

```
//function to lock the servo
void lock(){
   //check if currently unlocked
   if(!locked){
       //turn red led on and green led off
       red = 1;
       green = 0;
       //turn servo to locked
       locker.write(0.95);
       //play noise to signal incorrect code or lock button
       play_incorrect_sound();
       //set locked to true
       locked = true;
   }
   else{
       play_incorrect_sound();
   }
}

//function to unlock the servo
void unlock(){
   //check if currently locked
   if(locked){
       //turn green led on and red led off
       green = 1;
       red = 0;
       //change servo to unlocked
       locker.write(0.0);
       //play sound to signal correct sound or unlocked
       play_correct_sound();
       //set locked to false
       locked = false;
   }
   else{
       play_correct_sound();
   }
}

int main() {
   //initialize lock as locked
   lock();
   //char to store the key that was pressed
```

```
   char key;
   //flag to prevent multiple inputs when holding down the button
   int released = 1;
   lock();
   //begin loop
   while (true) {
       //initialize combination as an empty string
       string combo = "";
       // loop through until the combination has 4 digits
       while(combo.size() < 4) {
           //read the key as an input
           key = kpad.ReadKey();
           //if a key is not being pressed
           if(key == '\0'){
               //set flag to 1 indicating key was released and that we are ready to
register a new keypress
               released = 1;
           }
           //a key has been pressed
           else if(released == 1){
               //check if lock button is pressed
               if(key == '#'){
                   //set released flag to 0 to register key as currently being pressed
                   released = 0;
                   //break as input will always be incorrect
                   break;
               }
               combo += key;
               //set released flag to 0 to show that key is still pressed
               released = 0;
           }
       }

       // Check if the entered combination is correct
       if (combo == "1234") {
           unlock();
           printf("Unlocked\r\n");
       } else {
           lock();
           printf("Locked\r\n");
       }
```

```
        // Reset combo after checking or add logic to clear it
        combo = "";
    }
}
```

```
#include "mbed.h"

#ifndef Buzzer_H
#define Buzzer_H

#define NOTE_B0 31
#define NOTE_C1 33
#define NOTE_CS1 35
#define NOTE_D1 37
#define NOTE_DS1 39
#define NOTE_E1 41
#define NOTE_F1 44
#define NOTE_FS1 46
#define NOTE_G1 49
#define NOTE_GS1 52
#define NOTE_A1 55
#define NOTE_AS1 58
#define NOTE_B1 62
#define NOTE_C2 65
#define NOTE_CS2 69
#define NOTE_D2 73
#define NOTE_DS2 78
#define NOTE_E2 82
#define NOTE_F2 87
#define NOTE_FS2 93
#define NOTE_G2 98
#define NOTE_GS2 104
#define NOTE_A2 110
#define NOTE_AS2 117
#define NOTE_B2 123
#define NOTE_C3 131
#define NOTE_CS3 139
#define NOTE_DB3 139
#define NOTE_D3 147
#define NOTE_DS3 156
#define NOTE_EB3 156
```

```
#define NOTE_E3 165
#define NOTE_F3 175
#define NOTE_FS3 185
#define NOTE_G3 196
#define NOTE_GS3 208
#define NOTE_A3 220
#define NOTE_AS3 233
#define NOTE_B3 247
#define NOTE_C4 262
#define NOTE_CS4 277
#define NOTE_D4 294
#define NOTE_DS4 311
#define NOTE_E4 330
#define NOTE_F4 349
#define NOTE_FS4 370
#define NOTE_G4 392
#define NOTE_GS4 415
#define NOTE_A4 440
#define NOTE_AS4 466
#define NOTE_B4 494
#define NOTE_C5 523
#define NOTE_CS5 554
#define NOTE_D5 587
#define NOTE_DS5 622
#define NOTE_E5 659
#define NOTE_F5 698
#define NOTE_FS5 740
#define NOTE_G5 784
#define NOTE_GS5 831
#define NOTE_A5 880
#define NOTE_AS5 932
#define NOTE_B5 988
#define NOTE_C6 1047
#define NOTE_CS6 1109
#define NOTE_D6 1175
#define NOTE_DS6 1245
#define NOTE_E6 1319
#define NOTE_F6 1397
#define NOTE_FS6 1480
#define NOTE_G6 1568
#define NOTE_GS6 1661
#define NOTE_A6 1760
```

```
#define NOTE_AS6 1865
#define NOTE_B6 1976
#define NOTE_C7 2093
#define NOTE_CS7 2217
#define NOTE_D7 2349
#define NOTE_DS7 2489
#define NOTE_E7 2637
#define NOTE_F7 2794
#define NOTE_FS7 2960
#define NOTE_G7 3136
#define NOTE_GS7 3322
#define NOTE_A7 3520
#define NOTE_AS7 3729
#define NOTE_B7 3951
#define NOTE_C8 4186
#define NOTE_CS8 4435
#define NOTE_D8 4699
#define NOTE_DS8 4978
#define REST 0
#endif
```

A link to the video recording of our demonstration:
https://youtube.com/shorts/Id3XtuauKOA?feature=share

References/Bibliography

1. Hackster.io. “Arduino Controlled Servo Door Lock.” *Hackster.io*, Hackster, 28 Feb. 2022, www.hackster.io/raghavdaboss/arduino-controlled-servo-door-lock-1c2239.
2. Parallax Inc. “4x4 Matrix Membrane Keypad Datasheet.” *Sparkfun*, cdn.sparkfun.com/assets/f/f/a/5/0/DS-16038.pdf. Accessed 4 Nov. 2025.
3. *SG90 Servo Motor Datasheet*. Components 101, components101.com/sites/default/files/component_datasheet/SG90%20Servo%20Motor%20Datasheet.pdf. Accessed 11 Nov. 2025.
4. “STM32 Nucleo-64 Boards User Manual.” *Instructure.com*, STMicroelectronics, 2017, csusm.instructure.com/courses/42137/files/7966678. Accessed 11 Nov. 2025.
5. “Servo.”*Mbed*,os.mbed.com/teams/Biorobotics-Project/code/Servo//file/7b87b0244a27/Servo.h/. Accessed Nov. 2025.
6. “Keypad Class Reference.”*Mbed*, os.mbed.com/users/grantphillips/code/Keypad/docs/tip/classKeypad.html. Accessed Nov. 2025.
7. “Servo - Cookbook.” *Mbed*, os.mbed.com/cookbook/Servo#library. Accessed Nov. 2025.
8. “Matrix Keypad Pinouts.” Adafruit Learning System, Adafruit Industries, 2025, learn.adafruit.com/matrix-keypad/pinouts. Accessed Nov. 2025.
9. “Keypad Course Work.” Google Drive, drive.google.com/file/d/1GNPHv7n7byjghH2Mc2ZP-BoAR1MZ896w/view?usp=sharing. Accessed Nov. 2025